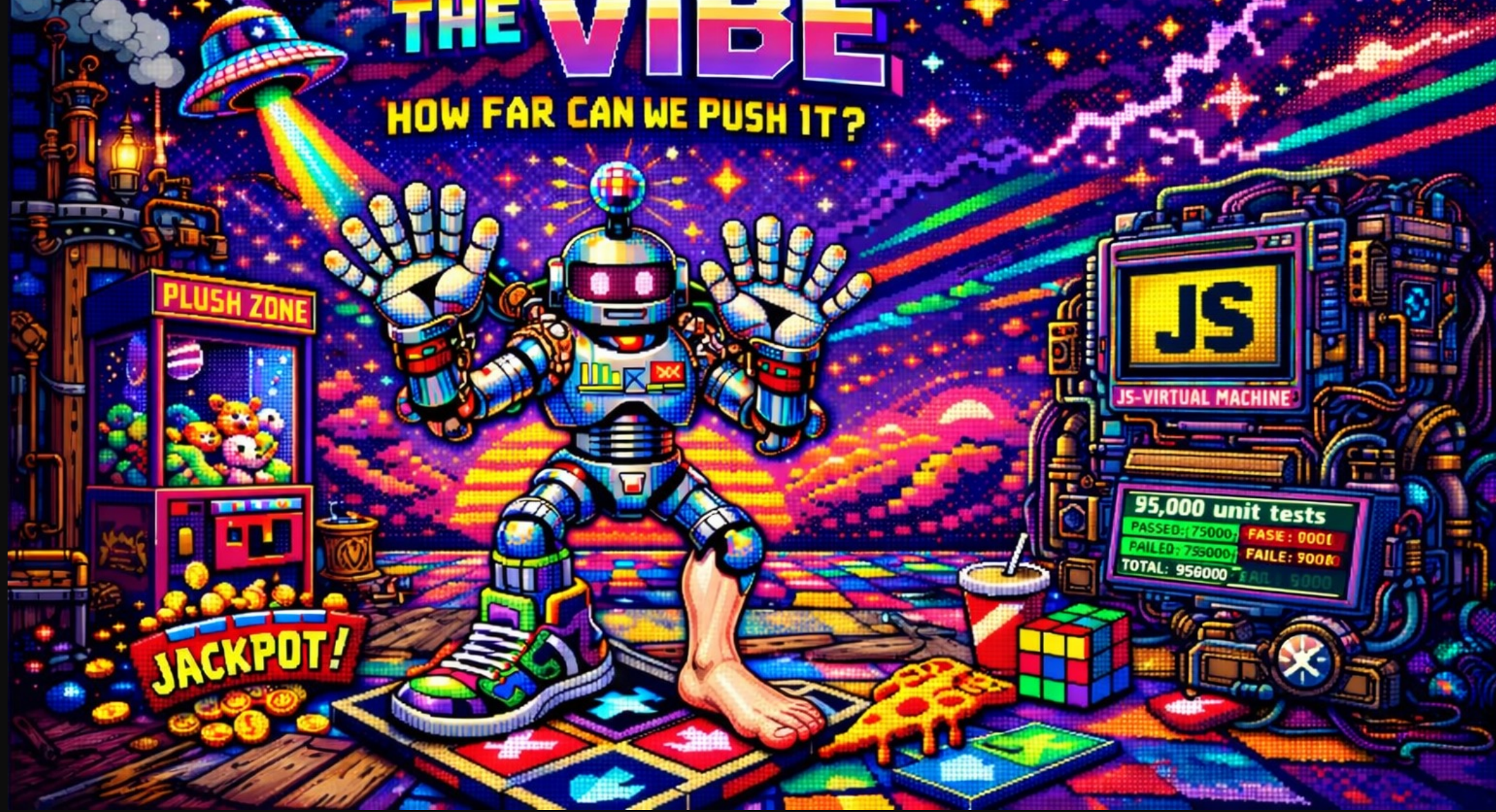


BEYOND THE VIBE

HOW FAR CAN WE PUSH IT?





Roger Johansson

Creator of:

- Proto.Actor
- Akka .NET
- Faktorial.AI
- Asynkron.JsEngine ← This talk

Company: Asynkron

Twitter: @rogeralsing

Github: rogeralsing

Mail: roger@asynkron.se

What to expect

- Hopefully convince some skeptics of what AI can accomplish
 - Building a JavaScript engine, the journey, not JavaScript as a language
 - Levels of agentic programming, from prompting to full automation
 - Scaling an AI project, process, tools, verification, and quality gates
 - Anecdotes, lessons learned, and key insights
-
- No absolute truths, just impressions. I'm probably wrong about a lot of this.

Part 1

Just vibing... no intent, no purpose

Rewind to November 2025



My profile

Connections 213

Following 1,152

Groups

Events

Saved items

Recent

frontend-developers

Show more ▾



Notifications

6



Derek Collins · 2nd

Senior Backend Engineer at LedgerFlow

4h · 🌐



Every time I test AI-generated code, it writes bad code, breaks edge cases, and creates more work.



96



Like



Comment



Repost



Jasmine Patel · 2nd

Frontend Developer at Brightside

5h · 🌐



You're an idiot if you believe AI can replace real developers.



158



Like



Comment



Repost



Marcus Lee · 2nd

Engineering Manager at HorizonPay

6h · 🌐



AI coding assistants are overhyped. They will never work for serious engineering teams.



74



Like



Comment



Repost



Dana Park · 2nd

Technical Lead at CloudCrest

7h · 🌐



LLMs don't understand architecture, scalability, or production systems.



110



Like



Comment



Repost

Why build a JS Engine?

...and why do it with 100% AI?

It started with this prompt

I want to create a JS execution engine using C#. It will need a js parser/grammar. I want to represent the result of the parser as **S-expressions / CDR/Cons**. The execution engine can then treat the code similar to **lisp**.

C# Cons.cs 5
M↓ context.md 6
C# Environment.cs 7
C# Evaluator.cs 8
C# HostFunction.cs 9
C# IJsCallable.cs 10
C# JsEngine.cs 11
C# JsFunction.cs 12
C# JsSymbols.cs 13
C# Lexer.cs 14
C# ParseException.cs 15
C# Parser.cs 16
C# ReturnSignal.cs 17
C# Symbol.cs 18
C# Token.cs 19
C# Foo 20
Dependencies 21
C# Program.cs 22
Scratches and Consoles 23

Run Foo x

🔄 📄 🗑️ ⋮

/Users/rogerjohansson/git/asynkron/OtelMCP/Asynkron.JsEngine/Foo/bin/Debug/net9.0/Foo

result: 111

```
var jsEngine = new JsEngine();
var code:Cons = jsEngine.Parse(source:@"
let x = 100;
function add(a, b) {
    function inner() {
        x = x + 1;
        return x;
    }

    return a + b+ inner();
}

add(5, 3);
add(5, 3);
add(5, 3); //returns this

");

var res:object? = jsEngine.Evaluate(code);
```

... 24 Hours later

- ✓ [📁] tests (192 tests) Success
 - ✓ [C#] Asyncron JsEngine.Tests (192 tests) Success
 - ✓ {} Asyncron JsEngine.Tests (192 tests) Success
 - ✓ AsyncAwaitTests (18 tests) Success
 - ✓ AsyncFunction_CanBeDeclared Success
 - ✓ **AsyncFunction_CanBeParsed** Success
 - ✓ AsyncFunction_ChainedCalls Success
 - ✓ AsyncFunction_HandlesRejection Success
 - ✓ AsyncFunction_NoReturn Success
 - ✓ AsyncFunction_ReturnsNull Success
 - ✓ AsyncFunction_ReturnsPromise Success
 - ✓ AsyncFunction_SequentialAwaits Success
 - ✓ AsyncFunction_WithAwait_ReturnsValue Success
 - ✓ AsyncFunction_WithAwaitInExpression Success
 - ✓ AsyncFunction_WithMultipleAwaits Success
 - ✓ AsyncFunction_WithTryCatch Success
 - ✓ AsyncFunctionExpression_CanBeParsed Success
 - ✓ AsyncFunctionExpression_Works Success
 - ✓ AwaitExpression_CanBeParsed Success
 - ✓ CpsTransformer_AlreadyTransformedAwaitDoesNotNeedTransformation Success
 - ✓ CpsTransformer_AlreadyTransformedCodeDoesNotNeedTransformation Success
 - ✓ CpsTransformer_TransformsIdempotent Success

```

1 class Calculator {
2   constructor(name) {
3     this.name = name;
4     this.history = [];
5   }
6
7   add(a, b) {
8     let result = a + b;
9     this.history.push(result);
10    return result;
11  }
12 }
13
14 // Generator function
15 function* fibonacci(n) {
16   let a = 0;
17   let b = 1;
18   let count = 0;
19   while (count < n) {
20     yield a;
21     let temp = a;
22     a = b;
23     b = temp + b;
24     count++;
25   }
26 }

```

Raw JavaScript

```

28 // Async function with await
29 async function fetchData(value) {
30   try {
31     let result = await Promise.resolve(value * 2);
32     let doubled = await Promise.resolve(result + 10);
33     return doubled;
34   } catch (error) {
35     return 0;
36   }
37 }
38
39 // Arrow function
40 let square = (x) => x * x;
41
42 // Main computation
43 let calc = new Calculator("MyCalc");
44 let sum = calc.add(5, 10);
45

```

```

(program
  (class Calculator null
    (lambda Calculator (name)
      (block
        (expr-stmt (set-prop this "name" name))
        (expr-stmt (set-prop this "history" (array))))
      ((method "add"
        (lambda null (a b)
          (block
            (let result (+ a b))
            (expr-stmt
              (call (get-prop (get-prop this "history") "push") result))
              (return result))))))
      (generator fibonacci (n)
        (block
          (let a 0)
          (let b 1)
          (let count 0)
          (while (< count n)
            (block
              (expr-stmt (yield a))
              (let temp a)
              (let temp a)
              (expr-stmt (assign b (+ temp b)))
              (expr-stmt (assign count (+ count 1)))))))
          (async fetchData (value)
            (block
              (try
                (block
                  (let result
                    (await (call (get-prop Promise "resolve") (* value 2))))
                  (let doubled
                    (await (call (get-prop Promise "resolve") (+ result 10))))
                  (return doubled))
                (catch error (block (return 0)))
                null)))
              (let square (lambda null (x) (block (return (* x x)))))
              (let calc (new Calculator "MyCalc"))
              (let sum (call (get-prop calc "add") 5 10)))

```

Translated to S-Expressions

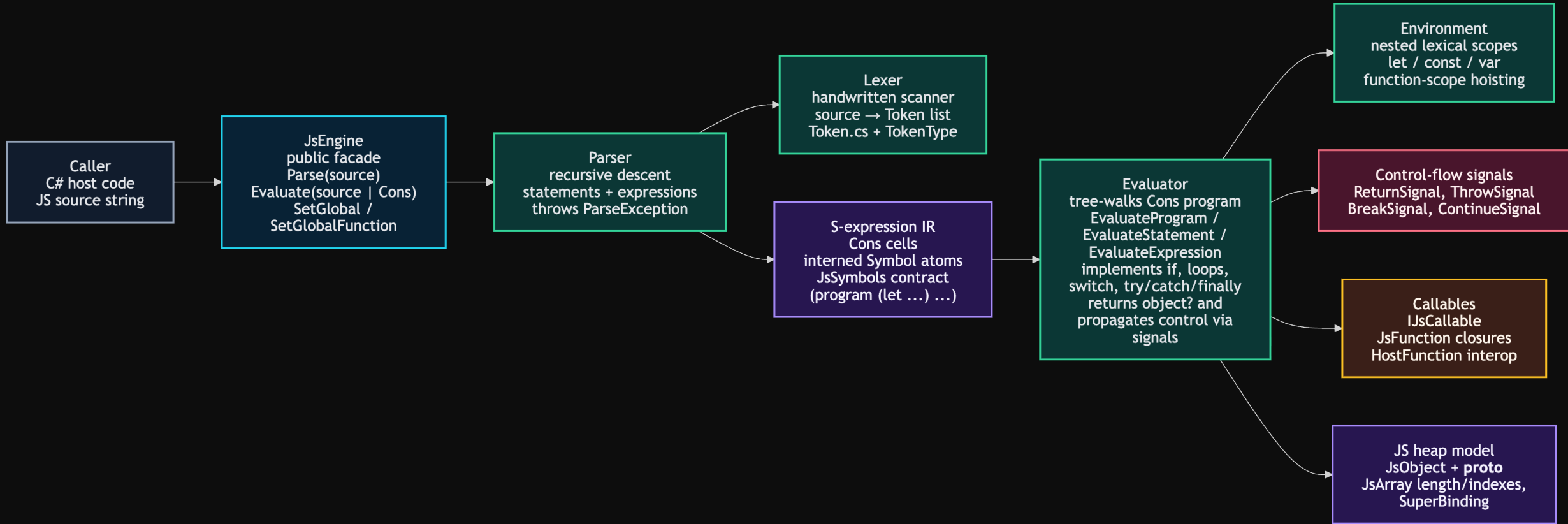
```

1 (program
2   (class Calculator null
3     (lambda Calculator (name)
4       (block
5         (expr-stmt (set-prop this "name" name))
6         (expr-stmt (set-prop this "history" (array))))
7       ((method "add"
8         (lambda null (a b)
9           (block
10            (let result (+ a b))
11            (expr-stmt
12              (call (get-prop (get-prop this "history") "push") result))
13              (return result))))))
14       (generator fibonacci (n)
15         (block
16           (let a 0)
17           (let b 1)
18           (let count 0)
19           (while (< count n)
20             (block
21               (expr-stmt (yield a))
22               (let temp a)
23               (let temp a)
24               (expr-stmt (assign b (+ temp b)))
25               (expr-stmt (assign count (+ count 1))))))
26           (function fetchData (value)
27             (block
28               (return
29                 (new Promise
30                   (lambda null (__resolve __reject)
31                     (block
32                       (try
33                         (block
34                           (try
35                             (block
36                               (expr-stmt
37                                 (call
38                                   (get-prop
39                                     (call (get-prop Promise "resolve") (* value 2))
40                                     "then")
41                                   (lambda null (result)
42                                     (block
43                                       (expr-stmt
44                                         (call
45                                           (get-prop
46                                             (call (get-prop Promise "resolve") (+ result 10))
47                                             "then")
48                                           (lambda null (doubled)
49                                             (block
50                                               (expr-stmt (call __resolve doubled))))))
51                                       (catch error
52                                         (block
53                                           (expr-stmt (call __resolve 0)))
54                                           null))
55                                       (catch __error
56                                         (block
57                                           (expr-stmt (call __reject __error)))
58                                           null))))))
59                               (let square (lambda null (x) (block (return (* x x)))))
60                               (let calc (new Calculator "MyCalc"))
61                               (let sum (call (get-prop calc "add") 5 10)))

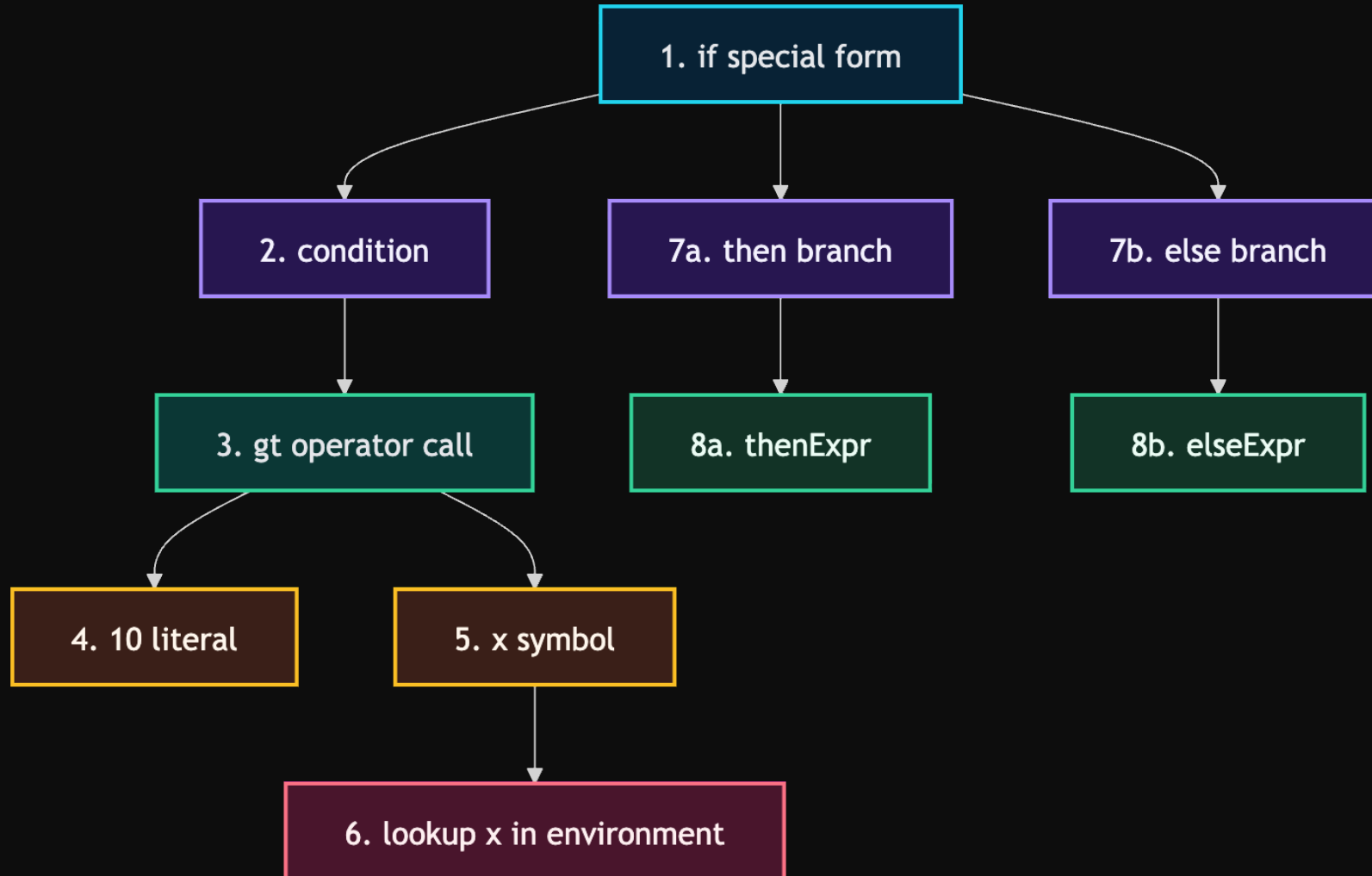
```

Continuation Passing Style

Architecture



Execution model



The initial itch, can I do this?

Adding more tests, design, architecture, trying to understand JS etc.

S-Expr to Typed AST

Memory models

Redesign Generators

Redesign Async Await

Improved Lexer/Parser

Runtime logs

The first steps towards a StdLib

Runtime spy – capture variables at given points

~1500 unit tests, including some known benchmark source files

... two weeks later I found it

ECMAScript Language Spec

ECMAScript Language Spec
94 000 unit tests

...How hard can it be?

ECMAScript Language Specification
94 000 unit tests

...Turns out, pretty darn hard

What the heck is AnnexB?

Sloppy vs Strict mode

Variable hoisting

Do people use legacy Octal literals?

There are RegEx tests for "Old Hungarian" and "Old South Albanian"?

Event loop, Tasks vs Microtasks

Module loading, sync, async

Truthy

Arrays with holes

Prototypal inheritance

The Standard Library - ~44 000 tests

Language Specs ~44 000 tests

How do you even run 94 000 tests?

...Can you even load this into your IDE?

94 000 tests and:

An immature runtime

Bugs

Infinite recursion

Memory explosion

Thread pool starvation

Circular dependencies, StdLib relies on working Language features, and vice versa

Language Tests

- For-loops
- Named function
- Anon function

Standard Library Tests

- Array.sort

AnnexB Tests

- ...

Temporal Tests

- ...

Still in pure vibe coding mode, absolutely no exciting insights yet
Still very undecided how far I want to take this
Maybe just full Language support?

Part 2

Scaling up, fast

**Too many tests,
Freezing tests,
Crashing tests,
IDE can't handle it
I'm not making progress**

...So I built a tool

...So I built a tool

Found 92945 tests

-Asynkron.JsEngine.Tests.Test262 (59413/92945)

49354 passed 9080 failed 847 skipped 5 hanging 127 crashed 04:36 (215,2/s)

54 %

```
* Temporal_PlainYearMonth_prototype_since("built-ins/Temporal/PlainYearMonth/prototype/since/roundingmode-halfExpand.js",True)
```

```
Object.prototype.__proto__ ("built-ins/Object/prototype/ __proto__ /set-immutable.js", File: 265
```

Asynchronous Test

```
Object.setPrototypeOf("built-ins/Object/setPrototypeOf/set-failure-cycle.js", false) 245
```

Process isolation

Process Isolation

```
! RegEx_propertyEscapes_generated("built-ins/RegExp/property-escapes/generated/Any.js",True) 13s
```

Multi-Platform

Multi-Process

Multi-Process.

Work pulling pattern

work building pattern.

from painting pattern

```
i: Temporal.PlainYearMonth.prototype[Symbol.species] = function (temporal/PlainYearMonth.prototype/since/roundingmode=halfTrunc.js", false)
```

Blame detector

Blaine detector.

Still some old Yunit/Munit

Still same old xunit/NUnit

```
Temporal.PlainYearMonth.prototype.since("built-in/Temporal/PlainYearMonth/prototype/since/roundingmode-invalid-string.js", True)
```

```
Temporal.Duration.from("built-ins/Temporal/Duration/from/negative-infinity-throws-rangeerror.js", true)
```

```

i: Temporal Duration from("built-ins/Temporal/Duration/from/non-integer-throws-rangeerror.is", False)

```

```
Temporal.Duration.from("built-ins/Temporal/Duration/from/non-integer-throws-rangeerror.js":True)
```

* Temporal.PlainYearMonth.prototype.since("built-ins/Temporal/PlainYearMonth/prototype/since/roundingmode-undefined.js", False)

**If the testrunner knows what
passed, failed, or crashed.
Why not let the testrunner
create the bug reports?**

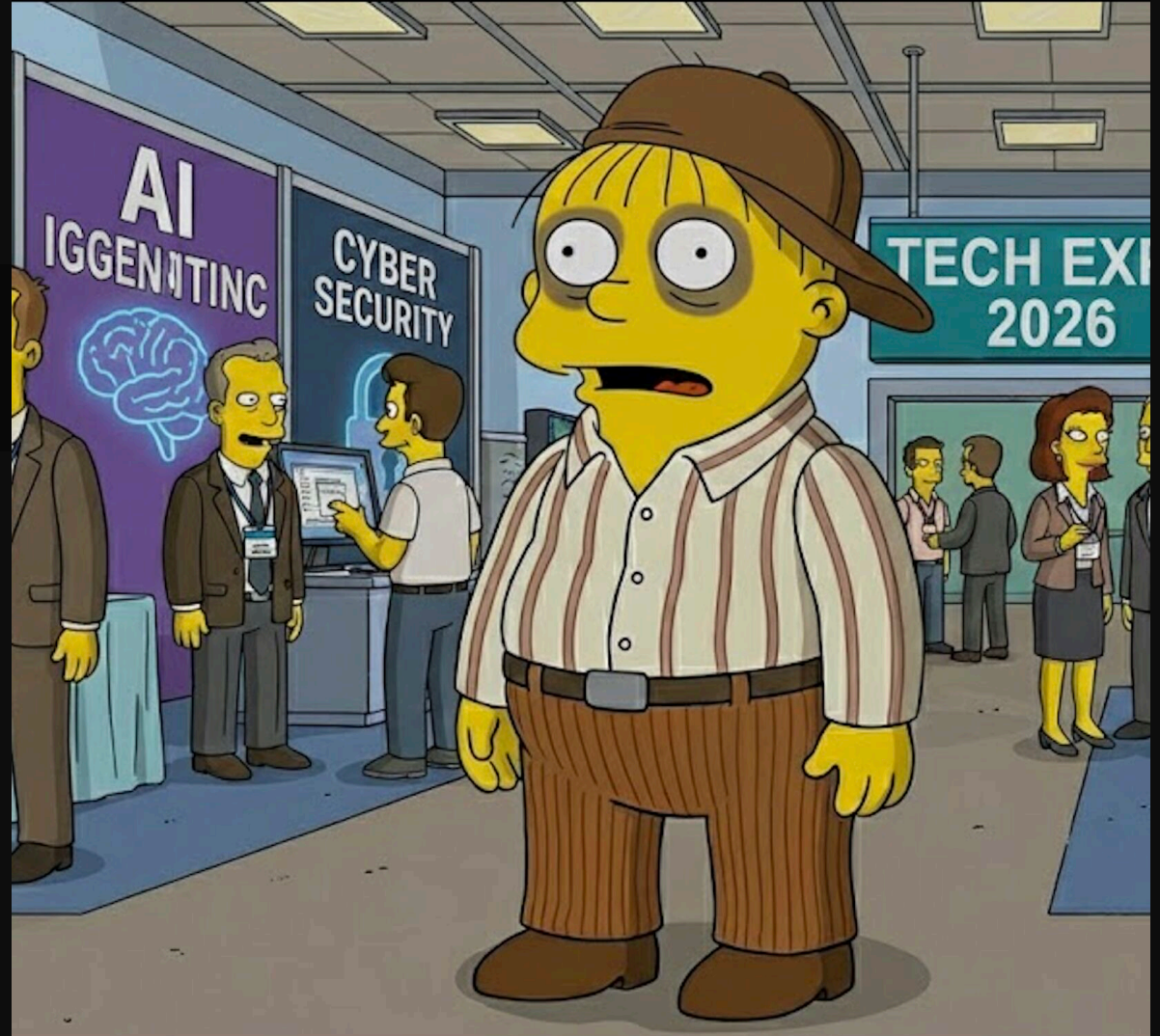
But...

**Now the same test runner
handed you a bug report with
80 000 failing tests...**

...So I built a tool

Not really, it was a Ralph loop

```
foreach bug in bugreports:  
    prompt = "Fix this bug..." + bug.Desc  
    RunAgent(prompt)
```



**Fixing bugs also means
uncovering bad design
...really bad design**

”Replaying generator”
...Three execution modes

AST Walking + CPS

IR Lowering

Replay Generator

AST Walking + ~~CPS~~

IR Lowering

Replay Generator



```
function* myGenerator() {  
    console.log("one");  
    yield 1;  
    console.log("two");  
    yield 2;  
    console.log("three");  
    yield 3;  
}
```

Output:

one

one

two

one

two

three

~~AST Walking + CPS~~

IR Lowering 👍

~~Replay Generator~~

Test Bomb

Layered Test



**You are almost
done!!**

Performance, where are all the bottlenecks or hotspots?

What about memory? Contention? Exceptions?

How do you ensure any form of code quality?

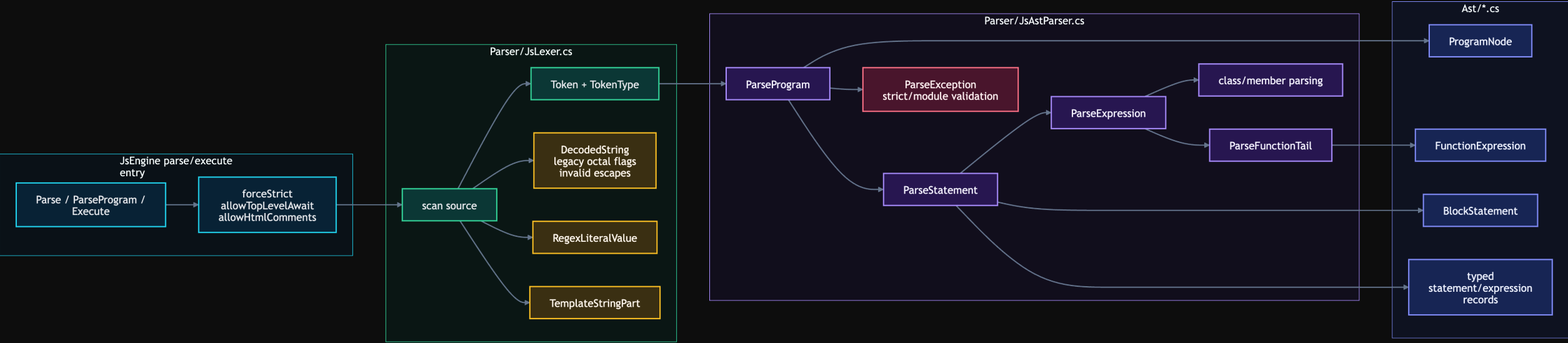
- C# take-no-prisoners. Warnings as errors, bump all Analyzers to error.
- How do we detect code duplication?

What about architecture, is the LISP model enough?

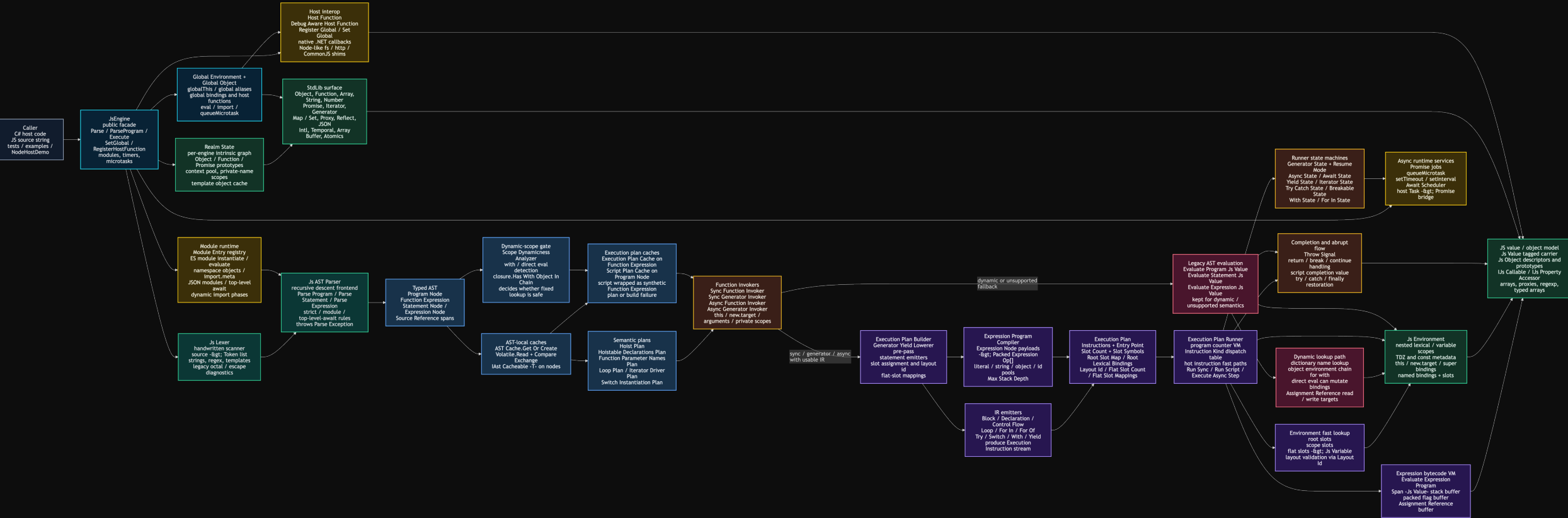
CPS, Generators, Replay, AST Eval, IR, ByteCode, IL?

Profiling

Frontend



"The mega flow"



I need something better...

I need something better...

Code duplication

I think the process is wrong

